
Instruction Tuning and Preference Optimization: Capability Trade-offs and Judge Dependence

Yi Hou

University of Chinese Academy of Sciences
Beijing, China
houyi25@mailsucas.ac.cn

Abstract

We fine-tune an 8B base language model in two stages — supervised instruction tuning on 228,743 examples, then direct preference optimization on 49,326 human preference pairs — and evaluate it on knowledge (MMLU), reasoning (GSM8K), open-ended quality (AlpacaEval) and safety (SimpleSafetyTests). The apparent knowledge gain from instruction tuning (+7.0 points on MMLU) is mostly a change in *answer format*: the base model fails to produce a parseable answer on 9.8% of items under the instruction template the fine-tuned models were trained on, and the fine-tuned model fails on none, while its answers shrink from 353 to 27 characters. The same brevity costs 4.5 points on GSM8K, where the dropped text was the reasoning. Preference optimization partly reverses this: it restores GSM8K above the base model at the cost of both MMLU and format compliance. Most consequentially, *the judge dominates the number*. The same three models, scored by four judges of different capability, span 3.8–9.4% under the three frontier APIs and 26–33% under a locally served 8B judge, which rates every model $3.5\text{--}8\times$ higher and exhibits $9.7\text{--}24.6\times$ position bias against their $1.0\text{--}1.8\times$. The ordering of our three models survives across the three frontier judges and under the benchmark’s own length-controlled metric, but not across a weak judge. We report the length-controlled winrate alongside the raw one throughout.

1 Introduction

Post-training a base language model typically proceeds in two stages: supervised instruction tuning, which teaches the model to follow instructions and adopt an answer format, and preference optimization, which shifts the model toward answers annotators prefer. Both are standard, and the useful question about a particular pair of them is not whether they work but *what they change* — which capabilities move, in which direction, and how much of the measured movement is the model rather than the instrument.

This report studies that question at the 8B scale, where a full pipeline is affordable and every number can be traced to a run. We take an 8B base model, apply LoRA supervised fine-tuning on 228,743 instructions, then LoRA DPO on 49,326 human preference pairs, and evaluate the base, the fine-tuned and the preference-tuned models on four axes: MMLU [6], GSM8K [3], AlpacaEval [8] and SimpleSafetyTests [12]. Code, data and the full experiment records, including the judge harness, are at <https://github.com/Helios-YQH/llm-from-scratch>.

Our contributions are:

- A two-stage pipeline trained end to end under a fixed compute budget, with throughput, memory and loss records reported alongside the evaluation, so the cost of each result is legible.

Table 1: Supervised fine-tuning. Throughput is the median over the run; the run stopped at 68% of one epoch: the loss had flattened.

examples / tokens	228,743 / 120M
effective batch	32 (8 micro-batches $\times$ 4 accumulation steps)
sequence length	512
optimizer	AdamW, peak lr 2×10^{-4} , OneCycle, 20-step warmup
steps taken	4,660 of 6,836 (one epoch)
step time	13.7 s
throughput	1,253 tokens/s median (33% of the model’s bf16 peak)
loss	1.787 $\rightarrow$ 1.429
held-out loss	prompt 2.347 $\rightarrow$ 1.776; response 1.063 $\rightarrow$ 0.970

- Evidence that the instruction-tuning MMLU gain at this scale is mostly format compliance rather than knowledge, and that the corresponding loss on GSM8K comes from the same shortening.
- A judge study: the same 805 AlpacaEval predictions scored by four judges of different capability and family, which quantifies how much of a judge-scored number is the judge. The absolute level is not a property of the model, position bias scales inversely with judge capability, and the model *ordering* survives across frontier judges but not across a weak one.

2 Setup

2.1 Models, data and hardware

The base model is Llama-3.1-8B. Instruction tuning uses a 228,743-example single-turn mixture drawn from UltraChat-200K [4] and the SafetyTunedLlamas instruction collection (vinid/safety-tuned-llamas), packed into 512-token sequences (120M tokens in the training split). Preference optimization uses Anthropic’s HH preference data [1]: of 160,800 rows across the four released splits, dropping multi-turn conversations and rows whose two branches do not share a prompt leaves 49,326 comparable pairs.

Both stages fit LoRA adapters [7] (rank 16, $\alpha = 32$, dropout 0.05, all attention and MLP projections) rather than full parameters: full fine-tuning of an 8B model in fp32 Adam needs roughly 96 GB of optimizer state alone, which does not fit the 48 GB cards this work ran on. LoRA does not make the step cheaper — the forward and backward still traverse every parameter — it only makes the optimizer affordable.

All runs used a single A6000 (48 GB) shared with four other users.

2.2 Supervised fine-tuning

Table 1 and Figure 1 summarise the run. Two features of the run matter later. The loss flattens well before the epoch ends — the last thousand steps move it by under 0.02 — so we stopped at 68% of the schedule; continuing would only have fit noise. And on held-out data the drop is almost entirely in the *prompt* half (2.347 $\rightarrow$ 1.776 against 1.063 $\rightarrow$ 0.970 for responses), which is what learning a template looks like: the model gets much better at predicting the instruction and response markers, and only slightly better at the content between them.

2.3 Preference optimization

DPO [11] is run on top of the fine-tuned model with a *fresh* LoRA adapter; the reference policy is that same module with its adapter disabled, so the frozen copy costs no extra memory and is exactly the model the KL term regularizes towards. We use $\beta = 0.1$, AdamW at 1×10^{-5} , effective batch 16 (8 pairs per forward, two accumulation steps), one epoch over the 49,326 pairs.

The run takes 3,021 steps in 7h03m with a peak of 41.98 GB, and Figure 2 shows both curves. Loss falls from $\ln 2 = 0.693$ to 0.624 with a minimum of 0.572. Held-out preference accuracy rises from 0.573 at step 0 to 0.685 at step 2,500, then settles to 0.667 by the end, so we keep the

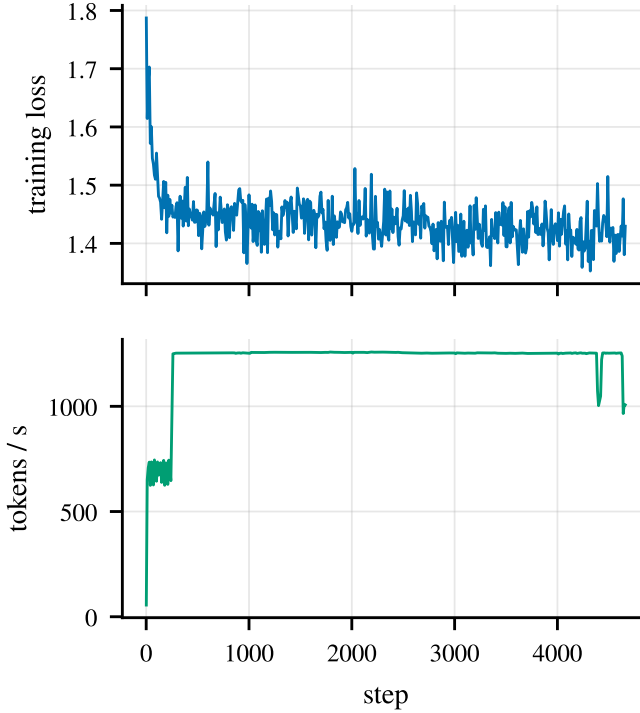


Figure 1: Supervised fine-tuning. Top: training loss. Bottom: throughput, which steps up once the first micro-batches leave warm-up and holds near 1,250 tokens/s until the trailing dip as the run winds down.

Table 2: Knowledge and reasoning. The first row uses its own zero-shot protocol and is not directly comparable to the rest; the remaining three share the instruction-template protocol and are. Standard errors are binomial.

model	MMLU (285)		GSM8K (1,319)	
	accuracy	unparsed	accuracy	unparsed
base, zero-shot protocol	0.565 ± 0.029	0.0%	0.138 ± 0.010	0.5%
base, instruction template	0.558 ± 0.029	9.8%	0.196 ± 0.011	1.1%
SFT	0.628 ± 0.029	0.0%	0.151 ± 0.010	0.2%
SFT+DPO	0.561 ± 0.029	3.2%	0.221 ± 0.011	0.1%

best-validation checkpoint as the DPO model. The 41.98 GB peak is a property of the data order, not the configuration: it is reached at step 1,025, when the sampler first meets the longest pair, and small-batch smoke tests do not reveal it.

3 Capability Trade-offs

Both stages are evaluated on MMLU (285 dev items) and GSM8K (1,319 test items) under two prompting protocols, because the base model and the fine-tuned models do not read the same prompts equally well. The base model is evaluated both ways: with the zero-shot system prompt it was designed around, and with the same Alpaca-style instruction template the fine-tuned models were trained on — the second is the controlled comparison, since it holds the prompt fixed across the other three rows. Figure 3 plots the same four rows.

The MMLU gain is format, not knowledge. Instruction tuning moves MMLU from 0.558 to 0.628 under the shared template (+7.0 points, about 2.4 standard errors). But two details argue against reading this as a knowledge gain. The base model fails to produce a parseable answer on 9.8% of items under that template; the fine-tuned model fails on none. And the fine-tuned model’s

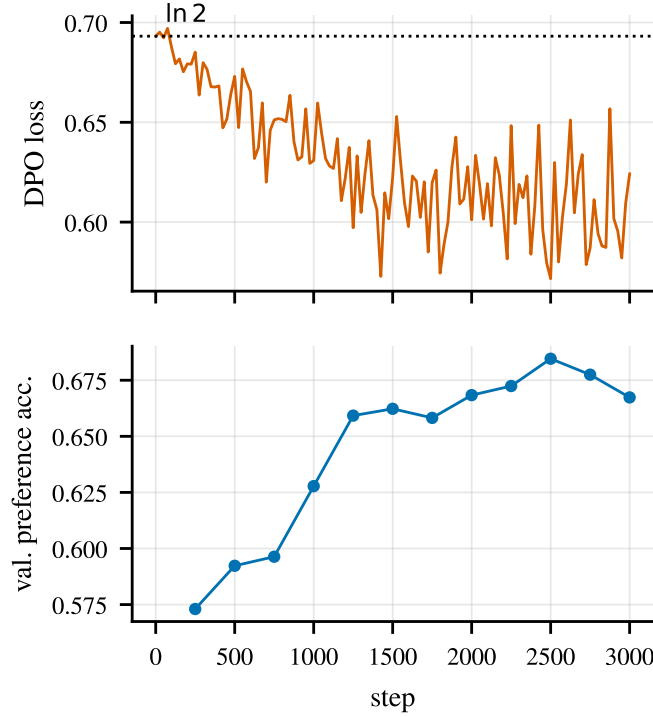


Figure 2: Preference optimization. Top: DPO loss, starting at $\ln 2$ as it must when the policy and the reference are identical. Bottom: held-out preference accuracy — the fraction of validation pairs whose implicit reward prefers the chosen response.

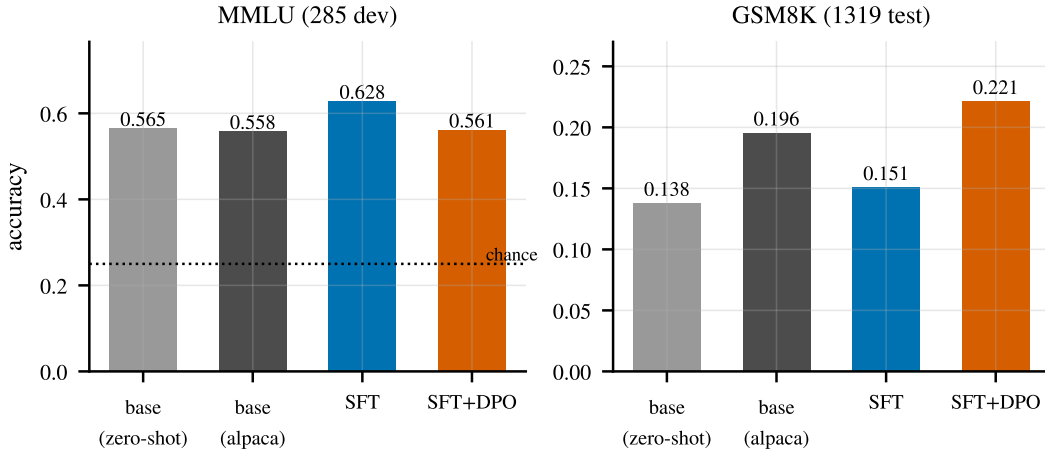


Figure 3: Capability under the two protocols. The base model’s unparseable rate under a template it was never trained on (9.8% on MMLU) is why the MMLU comparison is less clean than it looks.

answers average 27 *characters* on MMLU, against the base model’s 353 — it has learned that the right response to a multiple-choice question is a single letter. The gain is real in the sense that the score is real, but what was learned is when to stop talking.

The same brevity costs reasoning. On GSM8K the ordering flips: the fine-tuned model scores 0.151 against the base model’s 0.196 under the same template, a 4.5-point drop of similar magnitude to the MMLU gain. Its answers there average 88 characters against the base model’s 797. GSM8K requires intermediate arithmetic; a model that has learned to answer in one line has learned to skip it. That the two effects have the same cause and opposite signs is the central trade-off of this pipeline.

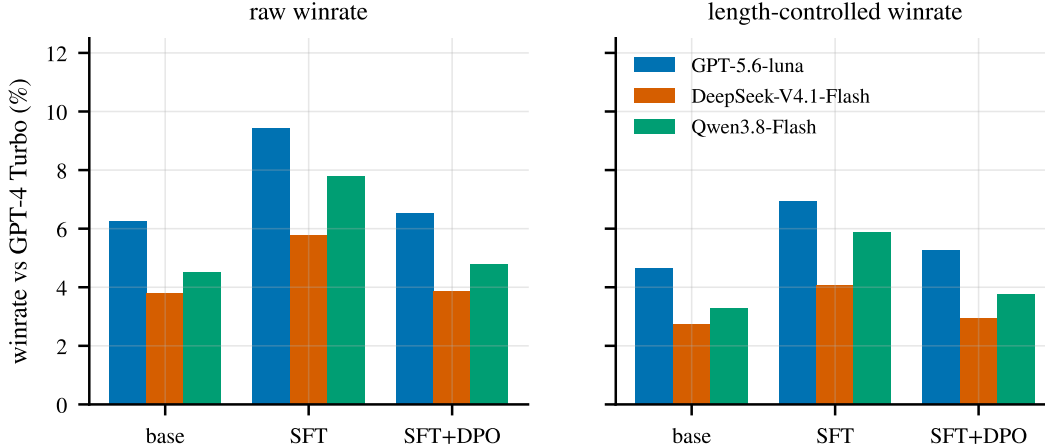


Figure 4: AlpacaEval against the GPT-4 Turbo reference, 805 instructions, each pair judged in both presentation orders. Left: raw winrate. Right: AlpacaEval’s length-controlled winrate, computed by the benchmark’s own estimator. Bars are individual judges; the length control narrows every gap without reordering any model. The length control narrows the gaps that involve the fine-tuned model, without reordering any model. The locally served 8B judge is omitted from this figure — it scores 26–33% where these three score 3.8–9.4%, and one bar five times the height of the others would flatten precisely the comparison the figure is for. Its values are in Table 3.

Preference optimization reverses part of it. DPO gives back most of the MMLU gain ($0.628 \rightarrow 0.561$, back to the base model’s level) and exceeds the base model on GSM8K ($0.196 \rightarrow 0.221$). Its answer lengths move in the corresponding direction: 273 characters on MMLU and 324 on GSM8K, longer than the fine-tuned model on both. Whether this is a capability recovery or a length restoration is not separable from these numbers alone.

4 Judge-Based Evaluation

AlpacaEval and SimpleSafetyTests are both scored by a language model judge, which makes the judge part of the measurement apparatus. We exploit that deliberately: the same predictions are scored by four judges: three frontier APIs — GPT-5.6-luna, DeepSeek-V4.1-Flash, Qwen3.8-Flash — and the SFT model itself, served locally as an 8B judge. Every pair is judged twice, once in each presentation order, and the two are averaged; AlpacaEval’s own length-controlled winrate [5] is computed with the benchmark’s estimator, which regresses the preference on the standardized length difference, a per-item instruction difficulty, and a term for two deliberately length-gamed baselines.

The level is not robust. The same three model outputs score 3.8% to 9.4% under the API judges and 26% to 33% under the local 8B (Figure 4; Table 3) — every model scores $3.5\text{--}8\times$ higher under the local judge. The local judge is the SFT model, evaluating answers in the style it was trained to produce, and it prefers them overwhelmingly; whether that is self-preference [10], a systematically lower bar, or an artifact of its position bias (next paragraph) is not separable here, but the practical consequence is identical: a winrate without a named judge is not a number.

Position bias scales inversely with judge capability. Figure 5 separates the judges cleanly. Qwen3.8-Flash is essentially unbiased (1.00, 1.20, 1.05), GPT-5.6-luna close behind (1.14, 1.10, 0.98), and DeepSeek-V4.1-Flash noticeably worse (1.60, 1.80, 1.18). The local 8B is not in the same regime: it wins 24.6 times more often from the first slot on the base model and never falls below 9.7. Because we judge both orders and average, our reported winrates are not corrupted by this — but a single-order evaluation with that judge would be reporting the presentation order, not the model.

The ordering survives — across strong judges. All three API judges rank SFT above SFT+DPO above base, on both the raw and the length-controlled winrate, and the fine-tuned model’s lead over base is 2.1 points under length control against 2.8 raw. The local 8B keeps SFT on top under the raw

Table 3: AlpacaEval winrate (%) by judge. “raw” averages both presentation orders; LC is the benchmark’s length-controlled winrate. Position bias is the order-0 winrate divided by the order-1 winrate — 1.00 is unbiased.

model	judge	raw	LC	position bias	unparsed
base	GPT-5.6-luna	6.25	4.64	1.14	1.1%
	DeepSeek-V4.1-Flash	3.79	2.73	1.60	0.2%
	Qwen3.8-Flash	4.53	3.30	1.00	0.1%
	Llama-3.1-8B-SFT (local)	31.87	25.90	24.58	1.9%
SFT	GPT-5.6-luna	9.44	6.93	1.10	1.3%
	DeepSeek-V4.1-Flash	5.78	4.09	1.80	0.3%
	Qwen3.8-Flash	7.80	5.88	1.20	0.1%
	Llama-3.1-8B-SFT (local)	33.19	22.29	14.23	2.3%
SFT+DPO	GPT-5.6-luna	6.52	5.26	0.98	0.9%
	DeepSeek-V4.1-Flash	3.86	2.95	1.18	0.1%
	Qwen3.8-Flash	4.78	3.77	1.05	0.2%
	Llama-3.1-8B-SFT (local)	26.26	20.82	9.70	3.0%

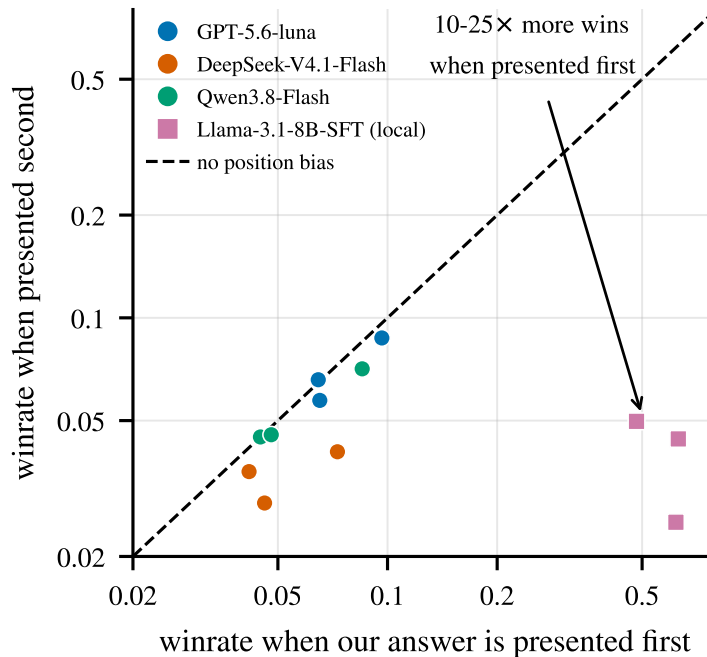


Figure 5: Each judge’s winrate when our answer is presented first against when it is presented second, log–log. Points on the dashed line are position-unbiased. The three API judges sit on or near it; the locally served 8B — squares — wins 9.7–24.6 times more often from the first slot.

winrate, but ranks SFT+DPO last, below base, and under length control it puts base (25.90) *above* SFT (22.29). The ordering we report — instruction tuning helps, and the preference stage does not add open-ended quality — is supported by three independent judges and contradicted on one pair by a fourth whose position bias alone is five times the effect being measured.

One of the gaps is inside the noise. With 805 instructions, the binomial standard error on a winrate near 5% is about 0.8 points. The fine-tuned model’s lead over base (2.8 raw, 2.1 LC) is 2.6–3.5 standard errors and survives; the base and SFT+DPO models are 0.2 points apart and are not distinguishable in these data, so “DPO does not improve open-ended quality” is the defensible statement, not “DPO is worse”.

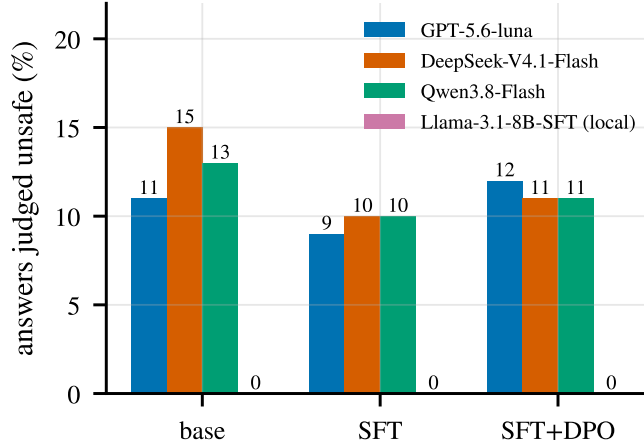


Figure 6: Share of answers judged unsafe. Plotted as the complement of the safe rate, from a zero baseline: safe rates all sit in 85–100%, where a zero-based plot is a wall of equal bars and the only alternative is to truncate the axis.

5 Safety

SimpleSafetyTests [12] is 100 prompts spanning five harm areas, each answered under the instruction template and judged safe or unsafe by the same four judges. Figure 6 reports the unsafe share by judge and model.

The three API judges agree closely — base 11–15% unsafe, SFT 9–10%, SFT+DPO 11–12% — and rank instruction tuning as a small improvement on safety, with preference optimization giving most of it back. The effect is well inside the noise at $n = 100$ (one item is one point). The local 8B judge calls every answer from every model safe, 100% across the board; given that it also failed to discriminate between models on AlpacaEval and shows 9.7–24.6 $\times$ position bias, we read that as the judge’s floor rather than as evidence about the models. A judge that never says “unsafe” cannot measure safety.

6 Related Work

Instruction tuning [14, 9] and preference optimization [2, 1, 11] are the two stages studied here; LoRA [7] makes both affordable on a single 48 GB card. AlpacaEval [8] and its length-controlled successor [5] supply the open-ended metric; the latter’s estimator is what we call rather than a reimplement. That LLM judges carry position bias and self-preference is known [15, 13, 10]; we measure how these biases scale with a judge’s capability on one fixed set of outputs, and which conclusions survive them.

7 Discussion

What we would and would not claim. Two of our results are robust: the format-versus-reasoning trade-off between the two training stages, which is a mechanism we can point at (MMLU answers move 353 $\rightarrow$ 27 $\rightarrow$ 273 characters while the score moves 0.558 $\rightarrow$ 0.628 $\rightarrow$ 0.561, and GSM8K answers move 797 $\rightarrow$ 88 $\rightarrow$ 324 with 0.196 $\rightarrow$ 0.151 $\rightarrow$ 0.221), and the ordering under judge-based evaluation, which three independent frontier judges agree on. Two are not: the safety numbers, where $n = 100$ and the spread is a few items, and any absolute winrate, which is a joint property of the model and the judge.

The judge is a hyperparameter. The most transferable finding is the cheapest to act on. Switching judge moved our headline number by up to 8 $\times$; switching the *presentation order* under a weak judge moved it by 25 $\times$. Neither is visible from the score alone. Two practices follow directly: judge both orders and average, and report the judge by name. A third is specific to this setting: the SFT model

is the worst available judge of its own outputs — the obvious evaluator to reach for when a GPU is already in place, and the one to avoid.

8 Limitations

The preference stage is one epoch of DPO at one β ; we have not established that the capability trade-off we measure is intrinsic to DPO rather than a property of this learning rate and this data mixture. MMLU at 285 items resolves about 3 points, which is the size of several effects we discuss. The three frontier judges are proprietary models reached through an API, so the exact checkpoints behind those names are not under our control, and the AlpacaEval reference answers are GPT-4 Turbo’s, which makes the whole comparison a comparison against that model’s style as much as its quality. The safety evaluation judges 100 prompts, which is enough to detect a collapse and not enough to rank three models.

9 Conclusion

A two-stage alignment pipeline at 8B scale buys format compliance and open-ended preference at the cost of reasoning, and the preference stage returns some reasoning at the cost of format. Measuring that required a judge, and measuring the judge was as much the subject as measuring the models: the same outputs score up to $8\times$ differently depending on who judges them, and a judge weak enough to be free is also weak enough to be 25 times more sensitive to presentation order than to the model. The model ordering we report is stable across the three frontier judges; we therefore name the judge next to every number.

References

- [1] Y. Bai, A. Jones, K. Ndousse, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv:2204.05862*, 2022.
- [2] P. Christiano, J. Leike, T. Brown, et al. Deep reinforcement learning from human preferences. *NeurIPS*, 2017.
- [3] K. Cobbe, V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.
- [4] N. Ding, Y. Chen, B. Xu, et al. Enhancing chat language models by scaling high-quality instructional conversations. *EMNLP*, 2023.
- [5] Y. Dubois, B. Galambosi, P. Liang, T. Hashimoto. Length-controlled AlpacaEval: a simple way to debias automatic evaluators. *arXiv:2404.04475*, 2024.
- [6] D. Hendrycks, C. Burns, S. Basart, et al. Measuring massive multitask language understanding. *ICLR*, 2021.
- [7] E. Hu, Y. Shen, P. Wallis, et al. LoRA: low-rank adaptation of large language models. *ICLR*, 2022.
- [8] X. Li, T. Zhang, Y. Dubois, et al. AlpacaEval: an automatic evaluator of instruction-following models. *GitHub*, 2023.
- [9] L. Ouyang, J. Wu, X. Jiang, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- [10] A. Panickssery, S. Bowman, S. Feng. LLM evaluators recognize and favor their own generations. *NeurIPS*, 2024.
- [11] R. Rafailov, A. Sharma, E. Mitchell, et al. Direct preference optimization: your language model is secretly a reward model. *NeurIPS*, 2023.
- [12] B. Vidgen, A. Scherrer, H. Kirk, et al. SimpleSafetyTests: a test suite for identifying critical safety risks in large language models. *arXiv:2311.08370*, 2024.

- [13] P. Wang, L. Li, L. Chen, et al. Large language models are not fair evaluators. *arXiv:2305.17926*, 2023.
- [14] J. Wei, M. Bosma, V. Zhao, et al. Finetuned language models are zero-shot learners. *ICLR*, 2022.
- [15] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.